

SPEC-1-OpenScholar-DDLC-Architecture

Background

OpenScholar is an open-access academic publishing platform aimed at removing barriers to research access and enabling student-first publishing. The PRD and SRS define a system that supports research submission, moderation, discovery, and engagement at scale. Given MVP constraints (fast delivery, limited resources, VPS deployment), the architecture must balance simplicity with future scalability toward a microservices ecosystem.

Requirements

Must Have

- User authentication (JWT-based)
- Paper submission with PDF upload ($\leq 50\text{MB}$)
- Metadata + version control
- Multi-author support with ordering
- Admin moderation workflow
- Search ($< 2\text{s}$ response)
- Comments and reactions
- View/download analytics
- Role-based access control

Should Have

- Notification system
- Dashboard analytics
- Multilingual support

Could Have

- Caching (Redis)
- Advanced analytics

Method

Architecture Style

Modular Monolith using Next.js (Fullstack)

- Single deployable unit
- Internal modular separation
- Future migration path to NestJS microservices

High-Level Architecture

- Frontend + API: Next.js
- Database: PostgreSQL
- Storage: MinIO (S3-compatible)
- Auth: JWT
- ORM: Prisma

Module Design

- Auth Module
- User Module
- Paper Module
- Author Module
- Search Module
- Engagement Module
- Moderation Module
- Analytics Module
- Notification Module
- Storage Module

Each module contains:

- service.ts (business logic)
- repository.ts (DB access via Prisma)

Request Flow

Client → API Route → Service Layer → Prisma → PostgreSQL

File Upload Flow

Client → API → MinIO → DB (store URL)

Search Design

- PostgreSQL Full-text search
- GIN index on keywords
- Future: Elasticsearch (optional)

Database Strategy

- Normalized schema
- Versioned papers
- Junction tables for relationships
- Soft delete for safety

API Design (Sample)

- POST /api/auth/register
- POST /api/papers
- GET /api/search
- POST /api/comments

Performance Strategy

- GIN index for keywords
- Aggregated metrics table
- Async analytics (future queue)

Migration Strategy

Modules are designed to be extractable:

- modules/paper → NestJS PaperModule
 - modules/auth → NestJS AuthModule
-

Implementation

1. Setup Next.js project
 2. Configure Prisma + PostgreSQL
 3. Implement modules (auth, paper, etc.)
 4. Integrate MinIO for file storage
 5. Build REST APIs
 6. Implement JWT authentication
 7. Add search with indexing
 8. Deploy to VPS using Docker/Nginx
-

Milestones

1. Week 1–2: Auth + DB setup
 2. Week 3–4: Paper submission + storage
 3. Week 5: Search + engagement
 4. Week 6: Admin moderation
 5. Week 7: Analytics + notifications
 6. Week 8: Deployment + testing
-

Gathering Results

Evaluate:

- Search response time (<2s)
- Upload success rate
- System uptime (99%)
- User engagement (comments/reactions)

Monitor:

- Query performance
- Storage usage
- Error rates

Need Professional Help in Developing Your Architecture?

Please contact me at www.devshahriar.com